

# UNIT I CLOUD AND SERVICES

**Introduction – Evolution of cloud computing – Hardware evolution – Internet software evolution – Server virtualization – Web services overview - IaaS - PaaS – SaaS - XaaS.**

## Topic 1 Evolution of Cloud Computing

### Introduction

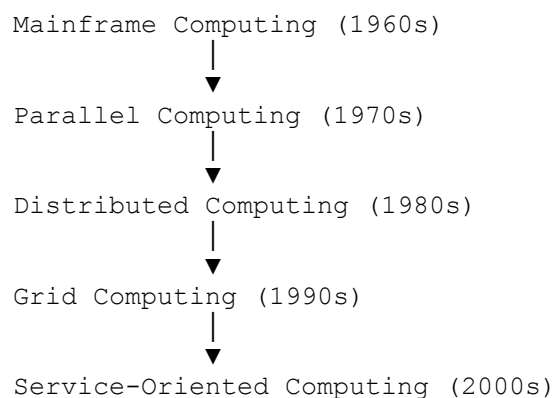
Cloud computing is **not a completely new technology**. It is the result of several decades of advancements in computing technologies.

Each stage addressed the limitations of the previous one and introduced new capabilities. The four major technologies that contributed to cloud computing are:

1. Parallel Computing
2. Distributed Computing
3. Grid Computing
4. Service-Oriented Computing (SOC)

These technologies eventually evolved into **Cloud Computing**.

### Evolution Timeline



↓  
▼

Cloud Computing (2006 onwards)

# Why Did Cloud Computing Evolve?

Earlier computing systems faced several challenges:

- Expensive hardware
- Limited processing power
- Poor scalability
- Resource underutilization
- Difficult maintenance
- Lack of flexibility

Each new technology attempted to solve one or more of these problems.

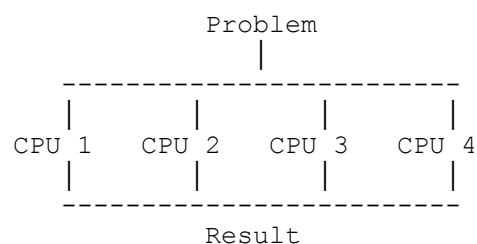
## 1. Parallel Computing

### Definition

Parallel computing is a computing model in which **multiple processors or cores execute different parts of the same program simultaneously** to solve a problem faster.

Instead of executing instructions sequentially, several computations occur at the same time.

### Architecture



### Characteristics

- Multiple processors work simultaneously.
- Shared memory architecture (in many systems).
- High-speed communication between processors.
- Suitable for computationally intensive applications.

### Advantages

- Faster execution
- High performance

- Better CPU utilization
- Reduced execution time

## Disadvantages

- Expensive hardware
- Complex programming
- Synchronization issues
- Limited scalability beyond a single system

## Applications

- Weather forecasting
- Scientific simulations
- Image processing
- Machine learning training
- Space research

## Example

Suppose a matrix multiplication task takes **100 seconds** on a single processor.

Using four processors:

```
Processor 1 → Part 1  
Processor 2 → Part 2  
Processor 3 → Part 3  
Processor 4 → Part 4
```

Total time  $\approx$  25 seconds

## Limitations of Parallel Computing

- Restricted to one computer.
- Cannot easily utilize remote resources.
- Hardware expansion is expensive.

These limitations led to **Distributed Computing**.

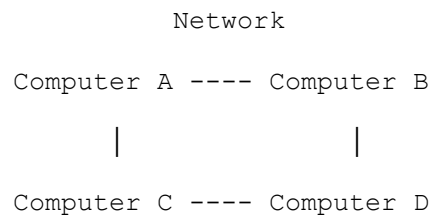
## 2. Distributed Computing

### Definition

Distributed computing is a system in which **multiple independent computers connected through a network work together to solve a problem**.

Each computer has its own processor and memory.

# Architecture



Each computer performs part of the computation.

## Characteristics

- Independent computers
- Connected by a network
- Resource sharing
- Fault tolerance
- Scalability

## Advantages

- Lower cost
- Easy scalability
- Improved reliability
- Resource sharing
- Better fault tolerance

## Disadvantages

- Network delays
- Security concerns
- Complex communication
- Data consistency issues

## Applications

- Banking systems
- Railway reservation systems
- Distributed databases
- Online multiplayer games
- E-commerce platforms

## Real-Life Example

Google Search

When a user searches for information:

- Thousands of servers process the request simultaneously.
- Results are combined and returned quickly.

## Difference Between Parallel and Distributed Computing

| Parallel Computing                | Distributed Computing |
|-----------------------------------|-----------------------|
| One computer                      | Multiple computers    |
| Multiple processors               | Multiple machines     |
| Shared memory                     | Separate memory       |
| High-speed internal communication | Network communication |
| Faster processing                 | Better scalability    |

## Limitations of Distributed Computing

Although distributed systems solved scalability problems, organizations still faced:

- Heterogeneous hardware
- Resource management difficulties
- Underutilized computers
- Difficult coordination

This led to **Grid Computing**.

## 3. Grid Computing

### Definition

Grid computing is a technology that **combines geographically distributed computers from different organizations to work together as a virtual supercomputer.**

Unlike distributed computing, the participating computers may belong to different institutions.

# Architecture

University A

|

Company B

|

Research Center C

|

Hospital D

|

Grid Middleware

|

Large Scientific Problem

## Characteristics

- Geographically distributed
- Different organizations
- Resource sharing
- Middleware-based management
- Heterogeneous systems

## Grid Middleware

Middleware acts as the coordinator.

Responsibilities include:

- Job scheduling
- Resource allocation
- Authentication
- Load balancing
- Monitoring

## Advantages

- Cost-effective
- Utilizes idle resources
- Massive computational power
- Supports large scientific applications

## Disadvantages

- Complex administration
- Network latency
- Security challenges
- Resource ownership issues

## Applications

- Drug discovery
- Genome analysis
- Climate modeling
- Particle physics (e.g., CERN)
- Aerospace research

## Example

Suppose 1,000 universities each contribute:

10 Computers

↓

Total = 10,000 Computers

↓

Acts as one supercomputer

## Why Grid Computing Was Not Enough

Grid computing still had several limitations:

- Difficult to configure
- No pay-per-use model
- Complex management
- Lack of virtualization
- Limited commercial adoption

These issues led to **Service-Oriented Computing**.

## 4. Service-Oriented Computing (SOC)

### Definition

Service-Oriented Computing is a computing paradigm where **software functionality is provided as reusable services that communicate over a network using standard protocols.**

Instead of building every application from scratch, developers can use existing services.

## Concept

Client Application

|

Web Services

|

Authentication Service

Payment Service

Email Service

Storage Service

Applications consume services as needed.

## Characteristics

- Loosely coupled services
- Reusable components
- Platform independent
- Standard communication protocols
- Service composition

## Key Technologies

- XML
- SOAP
- REST
- WSDL
- UDDI

## Advantages

- Code reuse
- Faster application development
- Easy integration
- Platform independence
- Better maintainability

## Disadvantages

- Network dependency
- Performance overhead
- Security concerns
- Service availability issues

## Example

An online shopping application may use:

- Authentication Service
- Payment Gateway
- Inventory Service
- Shipping Service
- Email Notification Service

Each service can be developed and maintained independently.

## How SOC Led to Cloud Computing

SOC introduced the idea of **delivering functionality as services** over the Internet. Cloud computing extended this idea by offering not only software services but also infrastructure and platforms on demand.

Examples:

- **Infrastructure as a Service (IaaS)** – Virtual machines, storage, networking.
- **Platform as a Service (PaaS)** – Development platforms and runtime environments.
- **Software as a Service (SaaS)** – Complete applications delivered through the web.

## Evolution

| Technology                 | Main Idea                                             | Limitation                                         |
|----------------------------|-------------------------------------------------------|----------------------------------------------------|
| Parallel Computing         | Multiple processors execute tasks simultaneously      | Limited to one machine                             |
| Distributed Computing      | Multiple computers cooperate over a network           | Difficult resource management                      |
| Grid Computing             | Shared resources across multiple organizations        | Complex management and no on-demand provisioning   |
| Service-Oriented Computing | Applications built from reusable services             | Focused mainly on software services                |
| Cloud Computing            | On-demand delivery of computing resources as services | Requires reliable Internet and security management |

# Comparison of Computing Paradigms

| Feature             | Parallel | Distributed | Grid    | Service-Oriented | Cloud                 |
|---------------------|----------|-------------|---------|------------------|-----------------------|
| Multiple processors | ✓        | ✓           | ✓       | ✗                | ✓                     |
| Multiple computers  | ✗        | ✓           | ✓       | ✓                | ✓                     |
| Internet-based      | ✗        | Partial     | ✓       | ✓                | ✓                     |
| Resource sharing    | Limited  | ✓           | ✓       | Service sharing  | Full resource sharing |
| Virtualization      | ✗        | ✗           | Limited | ✗                | ✓                     |
| Pay-as-you-go       | ✗        | ✗           | ✗       | ✗                | ✓                     |
| Scalability         | Medium   | High        | High    | High             | Very High             |

## Example

Imagine building a house:

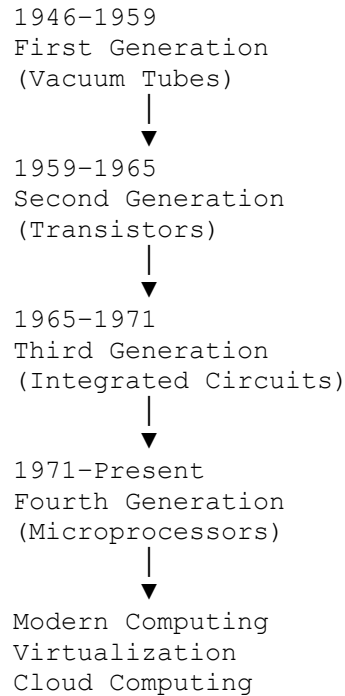
- **Parallel Computing:** Four workers paint different rooms at the same time.
- **Distributed Computing:** Different construction teams work on different sections of the house.
- **Grid Computing:** Teams from different companies collaborate on a large construction project.
- **Service-Oriented Computing:** Specialized contractors (electrician, plumber, painter) provide reusable services.
- **Cloud Computing:** You hire a construction company that provides workers, equipment, materials, and maintenance on demand, paying only for the services you use.

## Topic 2 : Hardware Evolution – Generations of Computers

### Introduction

The evolution of cloud computing is closely related to the evolution of computer hardware. Over the years, computers have become **smaller, faster, more reliable, energy-efficient, and cost-effective**. Each generation introduced significant technological improvements that eventually enabled virtualization, distributed computing, and cloud computing.

# Evolution Timeline



## What is a Computer Generation?

A **computer generation** refers to a stage in the evolution of computer technology based on major improvements in hardware components.

Each generation is identified by the primary electronic technology used to build computers.

## First Generation Computers (1946–1959)

### Technology Used

- Vacuum Tubes

Vacuum tubes were electronic devices used for switching and amplification.

### Architecture

Input

↓

Vacuum Tube Circuits

↓

Memory

↓

Output

Thousands of vacuum tubes were connected together to perform computations.

## **Characteristics**

- Vacuum tube technology
- Machine language programming
- Magnetic drum memory
- Very large size
- Consumed enormous electrical power
- Generated excessive heat
- Very expensive
- Low reliability
- Slow input/output operations

## **Advantages**

- First electronic computer generation
- Faster than mechanical computers
- Capable of solving complex mathematical problems

## **Disadvantages**

- Huge physical size
- High power consumption
- Frequent hardware failures
- Expensive maintenance
- Limited programming capability

## **Examples**

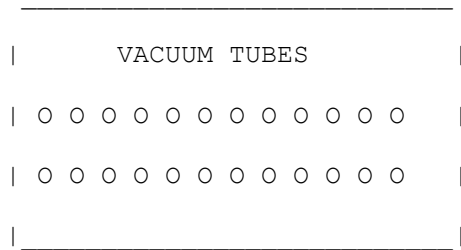
- ENIAC
- EDVAC
- EDSAC
- UNIVAC I
- IBM 701

## **Applications**

- Military calculations
- Census processing

- Scientific research
- Nuclear simulations

## Illustration



# Second Generation Computers (1959–1965)

## Technology Used

- Transistors

A transistor replaced the bulky vacuum tube.

## Why Were Transistors Better?

| Vacuum Tube            | Transistor            |
|------------------------|-----------------------|
| Large                  | Small                 |
| High heat              | Low heat              |
| High power consumption | Low power consumption |
| Less reliable          | More reliable         |
| Expensive              | Cheaper               |

## Characteristics

- Transistor technology
- Magnetic core memory
- Assembly language
- High reliability
- Smaller size
- Lower electricity consumption
- Improved processing speed

## Advantages

- Faster computation

- Lower maintenance
- More reliable
- Less heat generation
- Portable compared to first generation

## **Disadvantages**

- Still expensive
- Air conditioning required
- Limited memory

## **Examples**

- IBM 1401
- IBM 7094
- CDC 1604
- Honeywell 400

## **Applications**

- Business data processing
- Banking
- Engineering calculations
- Commercial organizations

## **Illustration**

Vacuum Tube

↓

Transistor

↓

Smaller Computer

↓

Better Performance

# **Third Generation Computers (1965–1971)**

## **Technology Used**

- Integrated Circuits (ICs)

Instead of using thousands of separate transistors, many transistors were fabricated on a single silicon chip.

## **What is an Integrated Circuit?**

An Integrated Circuit (IC) is a small semiconductor chip containing thousands of electronic components.

## **Characteristics**

- Integrated Circuit technology
- Operating systems introduced
- Multiprogramming
- Time-sharing systems
- Keyboard and monitor support
- Higher reliability
- Smaller size
- Lower cost

## **Advantages**

- Very high speed
- Reduced size
- Lower maintenance
- Less power consumption
- Increased storage capacity

## **Disadvantages**

- IC manufacturing was expensive
- Required sophisticated technology

## **Examples**

- IBM System/360
- PDP-8
- PDP-11
- Honeywell 6000

## **Applications**

- Universities
- Banking

- Research laboratories
- Airline reservation systems

## Illustration

Many Transistors

↓

Integrated Circuit (IC)

↓

Small Computer

↓

High Speed

# Fourth Generation Computers (1971–Present)

## Technology Used

- Microprocessor

A microprocessor integrates the CPU onto a single chip.

## What is a Microprocessor?

A microprocessor is a complete Central Processing Unit (CPU) fabricated on a single integrated circuit.

## Architecture

Keyboard

↓

Microprocessor

↓

Memory

↓

Storage

↓

## **Characteristics**

- Microprocessor technology
- Semiconductor memory
- Personal computers (PCs)
- Graphical User Interface (GUI)
- Networking support
- High-speed processing
- Very low power consumption
- Large storage capacity
- High reliability

## **Advantages**

- Very small size
- High processing speed
- Portable devices
- Low cost
- High storage capacity
- Easy maintenance
- Internet connectivity

## **Disadvantages**

- Rapid hardware obsolescence
- Increased security threats due to networking
- Dependence on software updates

## **Examples**

- IBM PC
- Apple Macintosh
- Intel Pentium-based computers
- Modern desktops and laptops

## **Applications**

- Education
- Healthcare
- Business
- Artificial Intelligence
- Internet services
- Cloud Computing
- Mobile Computing

## Illustration

Microprocessor

↓

Desktop

↓

Laptop

↓

Server

↓

Cloud Data Center

## Comparison of Computer Generations

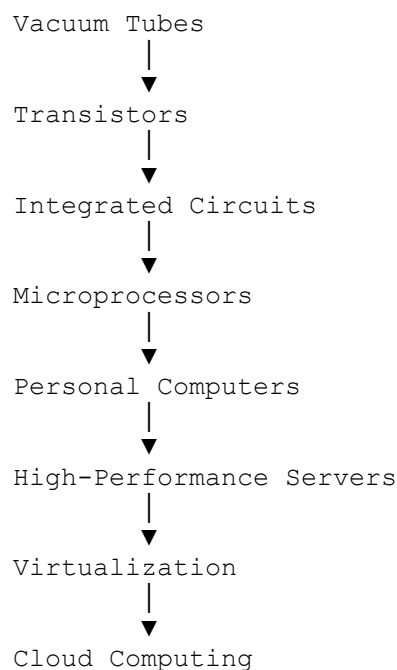
| Feature              | First Generation | Second Generation | Third Generation     | Fourth Generation                      |
|----------------------|------------------|-------------------|----------------------|----------------------------------------|
| Time Period          | 1946–1959        | 1959–1965         | 1965–1971            | 1971–Present                           |
| Technology           | Vacuum Tubes     | Transistors       | Integrated Circuits  | Microprocessors                        |
| Programming Language | Machine Language | Assembly Language | High-Level Languages | High-Level & Object-Oriented Languages |
| Size                 | Very Large       | Smaller           | Compact              | Very Small                             |
| Speed                | Slow             | Faster            | High                 | Very High                              |
| Reliability          | Low              | Moderate          | High                 | Very High                              |
| Power Consumption    | Very High        | High              | Moderate             | Low                                    |
| Heat Generation      | Very High        | Moderate          | Low                  | Very Low                               |
| Cost                 | Very Expensive   | Expensive         | Moderate             | Affordable                             |
| Memory               | Magnetic         | Magnetic          | Semiconductor        | Semiconductor                          |

|          |                 |          |                         |                         |
|----------|-----------------|----------|-------------------------|-------------------------|
|          | Drum            | Core     | Memory (early adoption) | Memory                  |
| Examples | ENIAC, UNIVAC I | IBM 1401 | IBM System/360          | IBM PC, Apple Macintosh |

## Contribution of Each Generation to Cloud Computing

| Generation        | Major Contribution                                                                    |
|-------------------|---------------------------------------------------------------------------------------|
| First Generation  | Introduced electronic computing                                                       |
| Second Generation | Improved reliability and reduced size                                                 |
| Third Generation  | Introduced operating systems, multiprogramming, and integrated circuits               |
| Fourth Generation | Enabled personal computers, networking, virtualization, Internet, and cloud computing |

## How Hardware Evolution Led to Cloud Computing



### Explanation:

- Faster processors enabled efficient execution of complex applications.
- Increased memory and storage supported large-scale data processing.

- Miniaturization reduced hardware costs and improved portability.
- Networking capabilities allowed computers to communicate globally.
- Powerful multi-core processors and high-capacity servers became the foundation of cloud data centers.
- Virtualization technology allowed multiple virtual machines to run on a single physical server, maximizing resource utilization and enabling cloud service models such as IaaS, PaaS, and SaaS.

## Summary

- The evolution of computer hardware progressed from **Vacuum Tubes** → **Transistors** → **Integrated Circuits** → **Microprocessors**.
- Each generation improved **speed, reliability, storage capacity, and energy efficiency**.
- Hardware advancements enabled operating systems, networking, virtualization, and distributed computing.
- Modern cloud computing relies on powerful fourth-generation hardware deployed in large-scale data centers.

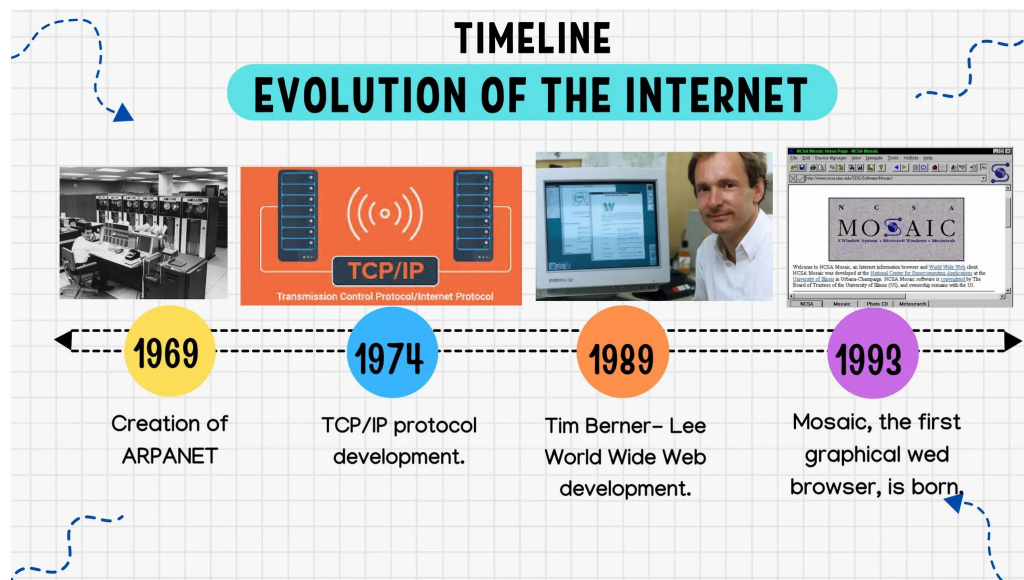
# Topic 3 Lecture Notes: Internet Software Evolution

## 1. Introduction

The Internet did not become a global communication platform overnight. In the early days, different computer networks used different communication methods, making it difficult for systems to exchange data.

The evolution of Internet software involved two major developments:

- Establishing common communication protocols so all networks could communicate.
- Building a common user interface that allowed ordinary people to access Internet resources easily.



## 2. Why Common Protocols Were Needed

During the 1960s and 1970s:

- Universities had their own networks.
- Research laboratories used different hardware.
- Government agencies used incompatible communication systems.
- Computers could not easily exchange data.

Problem:

A computer on one network could not reliably communicate with a computer on another network.

Solution:

Create a standard set of communication rules called protocols.

## 3. Establishing Common Protocols for the Internet

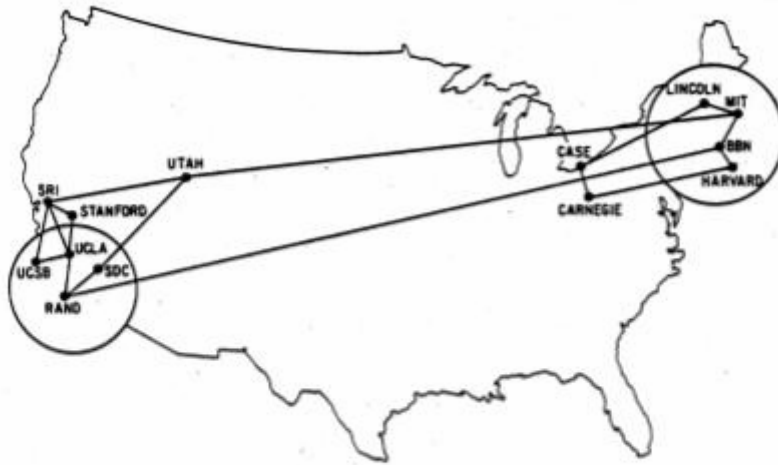
### What is a Protocol?

A protocol is a set of rules that defines how data is transmitted between devices.

Examples:

- HTTP – Web communication
- FTP – File transfer
- SMTP – Email transfer
- TCP/IP – Core Internet communication

### ARPANET – The Beginning



*ARPANet (1971), connecting 13 Defense research institutions and Universities.*

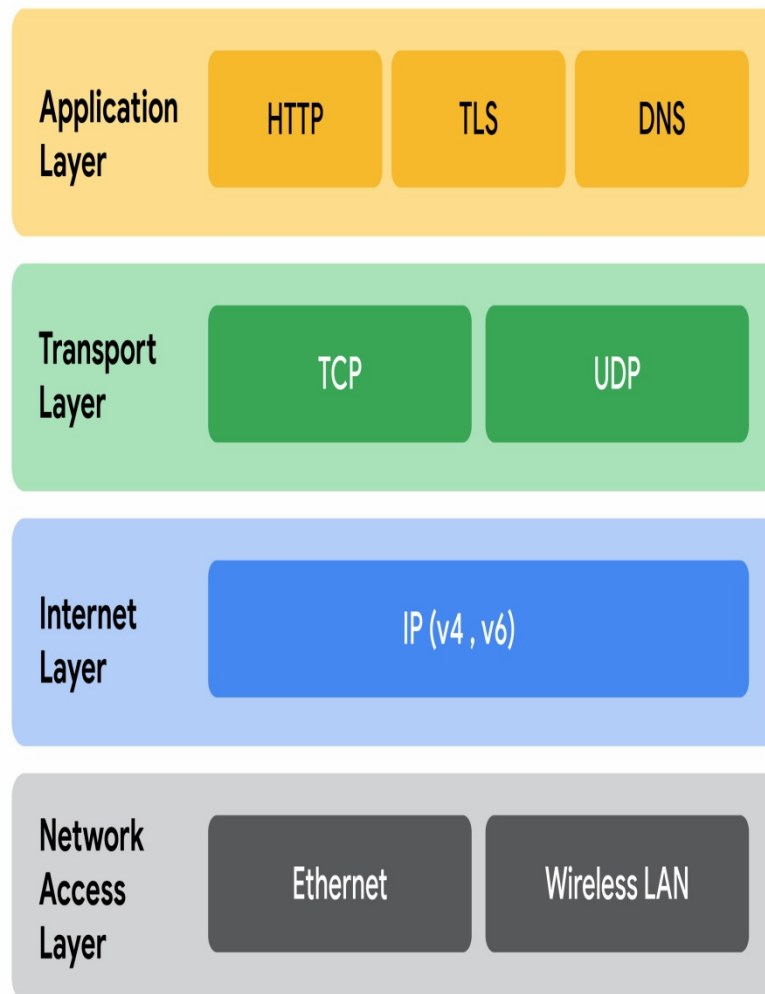
ARPANET (1969) was the first major packet-switched network developed by the U.S. Department of Defense.

Its goal was to connect research institutions and allow resource sharing.

### **TCP/IP Protocol Suite**

In the 1970s, Vinton Cerf and Robert Kahn developed TCP/IP.

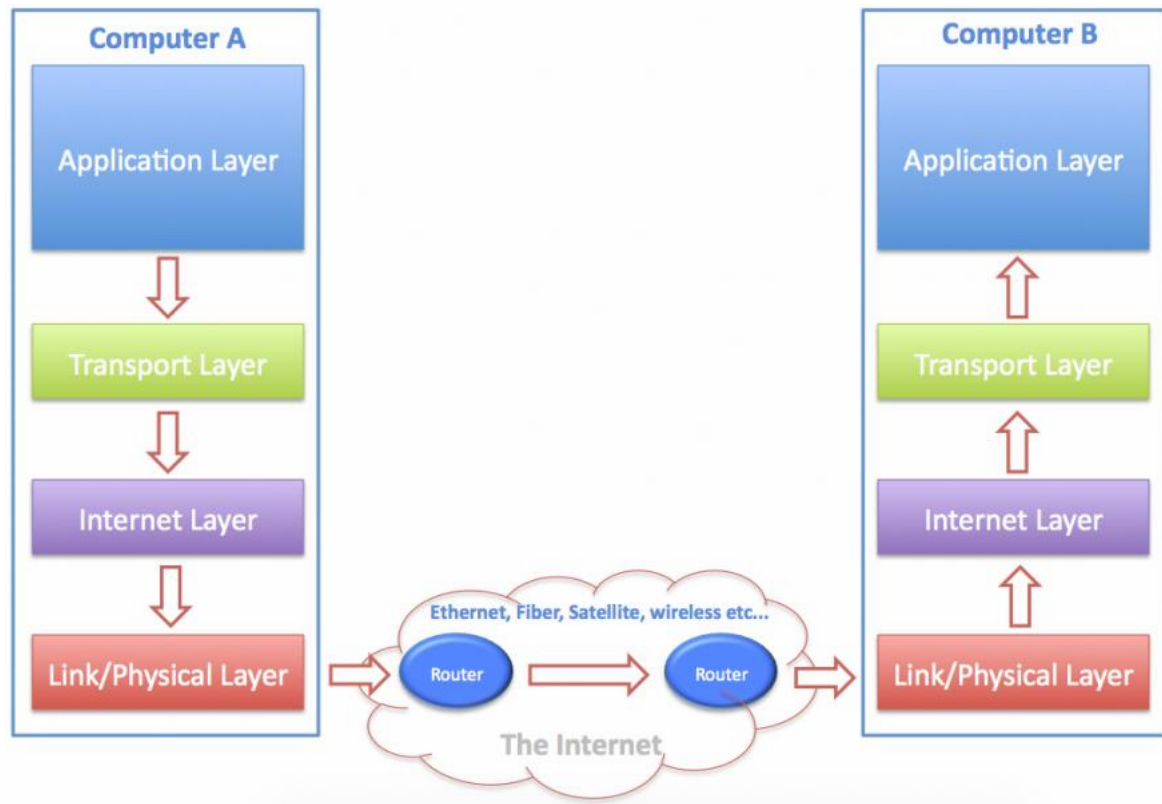
TCP/IP became the universal protocol of the Internet.



## Functions of TCP/IP

| Protocol | Function                             |
|----------|--------------------------------------|
| TCP      | Reliable data delivery               |
| IP       | Addressing and routing               |
| UDP      | Fast but less reliable communication |
| HTTP     | Web page transfer                    |
| FTP      | File transfer                        |

## How TCP/IP Works



Step-by-step:

- Application creates data.
- TCP divides data into packets.
- IP adds destination address.
- Routers forward packets through the Internet.
- Receiver reassembles the data.

#### 4. Evolution from IPv4 to IPv6

##### What is an IP Address?

An IP address uniquely identifies a device on a network.

##### IPv4

IPv4 was introduced in the 1980s.

Format:

192.168.1.1

Address Size: 32 bits

Total Addresses:

$2^{32} \approx 4.3$  billion addresses

## Problem with IPv4

As the Internet grew:

- Computers increased.
- Smartphones appeared.
- IoT devices expanded rapidly.
- Cloud services required millions of addresses.

Result: IPv4 addresses became insufficient.

## IPv6

IPv6 was developed to solve address exhaustion.

### IPv6 Address Example

2001:0db8:85a3:0000:0000:8a2e:0370:7334

### Key Features of IPv6

| Feature       | IPv4        | IPv6                   |
|---------------|-------------|------------------------|
| Address Size  | 32 bits     | 128 bits               |
| Address Count | 4.3 billion | 340 undecillion        |
| Security      | Optional    | Built-in IPsec support |
| Configuration | Manual/DHCP | Auto-configuration     |
| Efficiency    | Lower       | Higher                 |

### Why IPv6 is Important for Cloud Computing

Cloud providers manage:

- Millions of servers
- Virtual machines
- Containers
- IoT devices
- Mobile users

IPv6 provides enough addresses for future cloud infrastructure.

## 5. Building a Common Interface to the Internet

### Early Internet Access

Before the World Wide Web, users accessed the Internet using:

- Command-line tools
- FTP commands

- Telnet
- Email clients

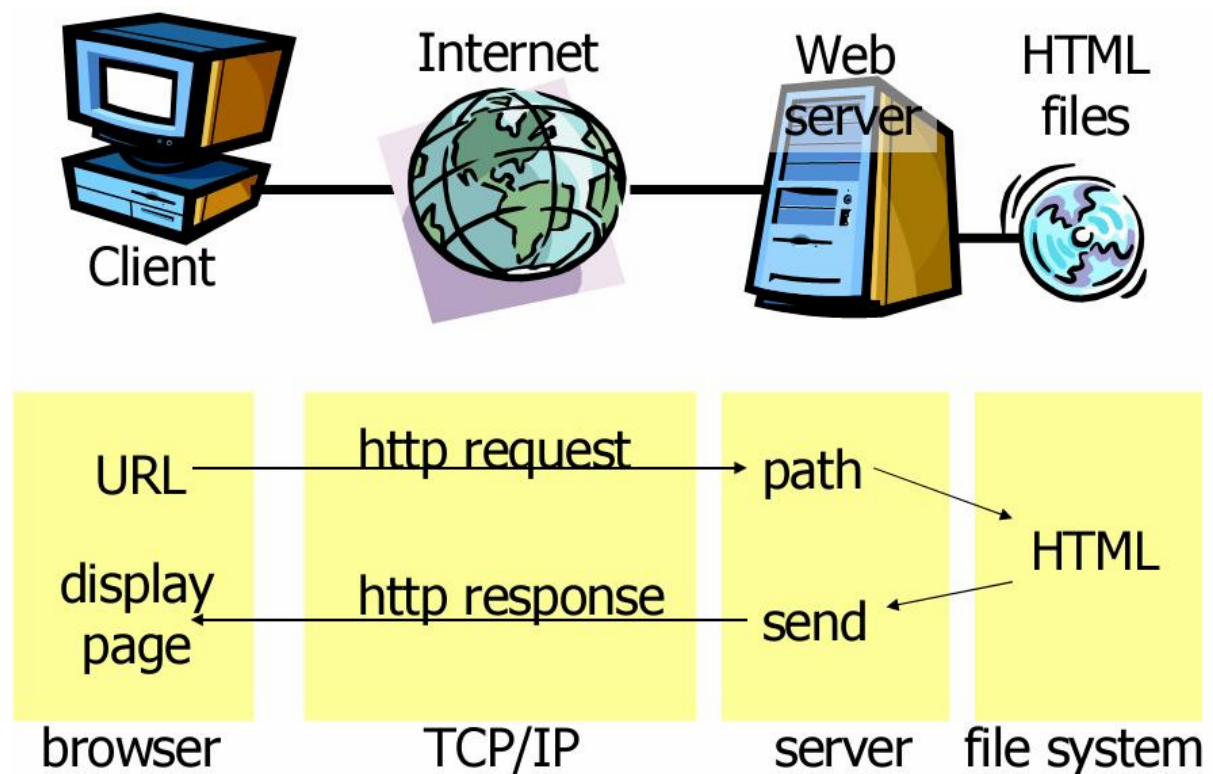
This was difficult for ordinary users.

## The World Wide Web (WWW)

In 1989, Tim Berners-Lee introduced:

- HTML – HyperText Markup Language
- HTTP – HyperText Transfer Protocol
- URL – Uniform Resource Locator

This created a common interface for accessing information.



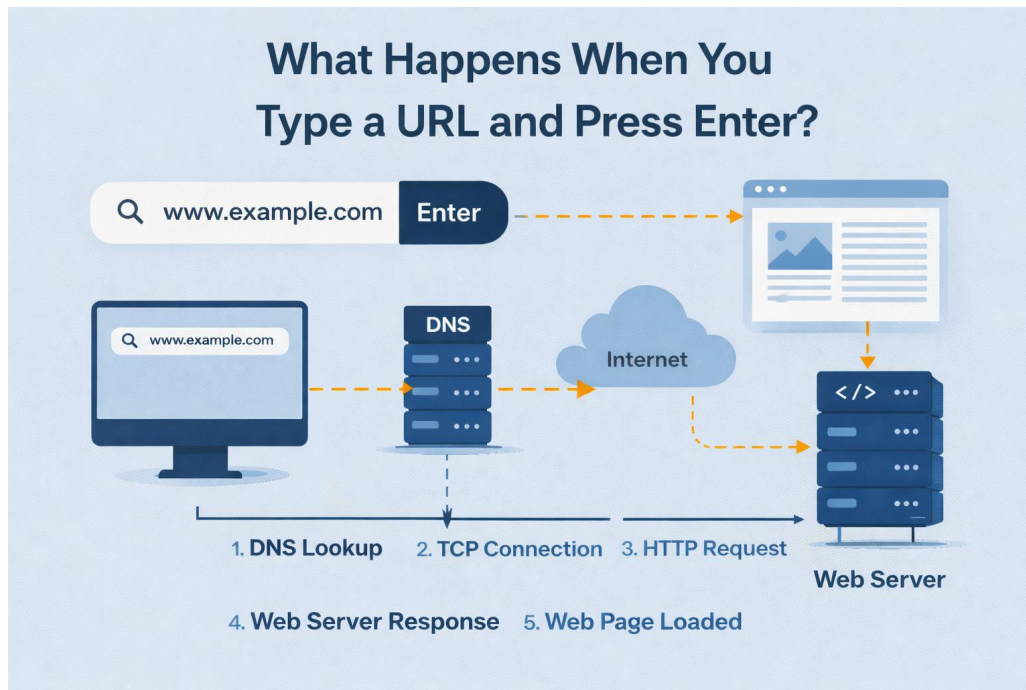
## Web Browser – The Common Interface

A web browser became the universal gateway to the Internet.

Examples:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

## How a Browser Accesses a Website



Process:

- User enters a URL.
- DNS converts the name to an IP address.
- Browser sends an HTTP request.
- Web server responds with HTML.
- Browser displays the webpage.

## Evolution of Internet Interfaces

| Stage          | Interface                    |
|----------------|------------------------------|
| Early Internet | Command Line                 |
| FTP/Telnet Era | Text-based tools             |
| World Wide Web | HTML Pages                   |
| Modern Web     | Interactive Web Applications |
| Cloud Era      | Browser + Mobile Apps + APIs |

## 6. Connection Between Internet Evolution and Cloud Computing

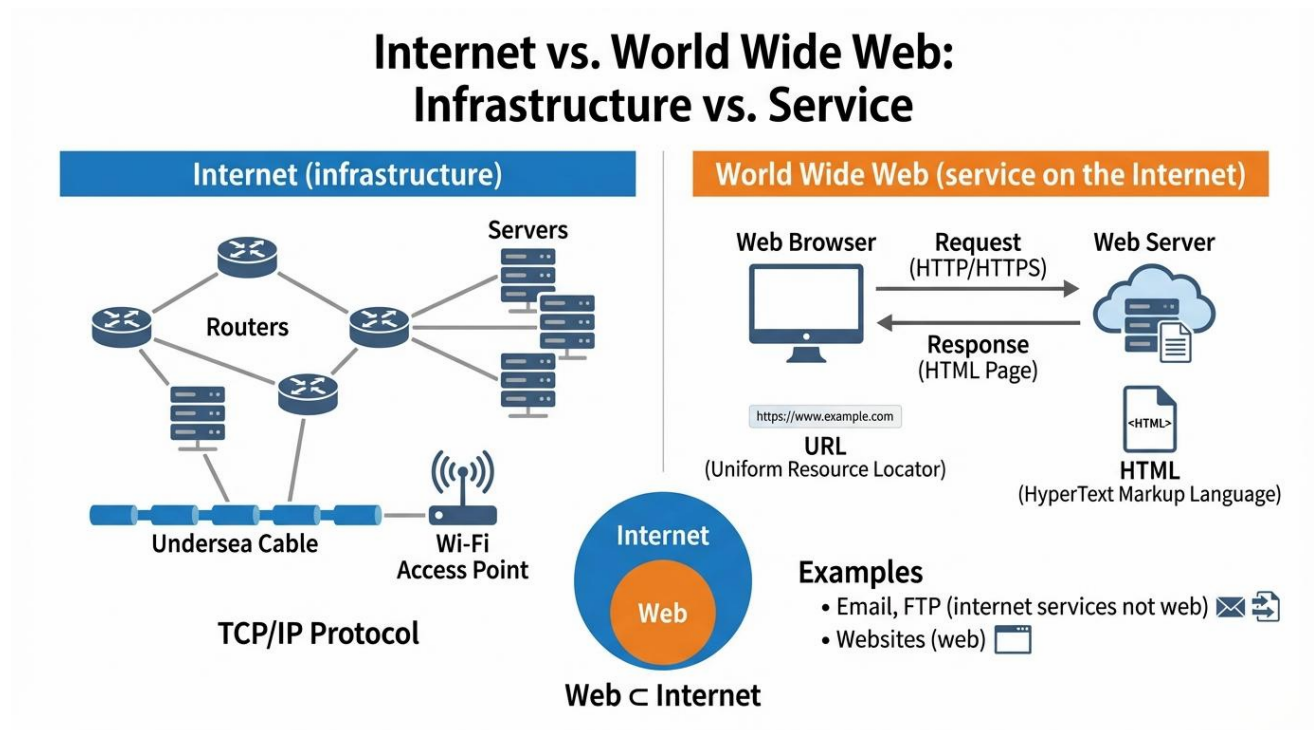
Cloud computing depends heavily on Internet software evolution.

Without:

- TCP/IP → devices cannot communicate.
- IPv6 → large-scale cloud addressing becomes difficult.
- HTTP/HTTPS → cloud services cannot be accessed through browsers.
- Web browsers → users cannot easily interact with cloud applications.

Therefore, Internet software evolution is the foundation of cloud computing.

## 7. Summary Diagram



### Summary

- The Internet required common protocols to connect different networks.
- TCP/IP became the universal communication standard.
- IPv4 provided Internet addressing but had limited address space.
- IPv6 solved address exhaustion and supports future cloud growth.
- The World Wide Web introduced HTML, HTTP, and URLs.
- Web browsers created a common interface for Internet access.
- Modern cloud services rely on all these Internet technologies.

## Topic 4 Server Virtualization and High-Performance Server Architectures

### 1. Introduction

As computer applications became more complex, a single processor was no longer sufficient for handling large-scale computations.

Applications such as:

- Weather forecasting
- Artificial Intelligence
- Scientific simulations
- Banking

- Big Data analytics
- Cloud computing

required much higher processing power.

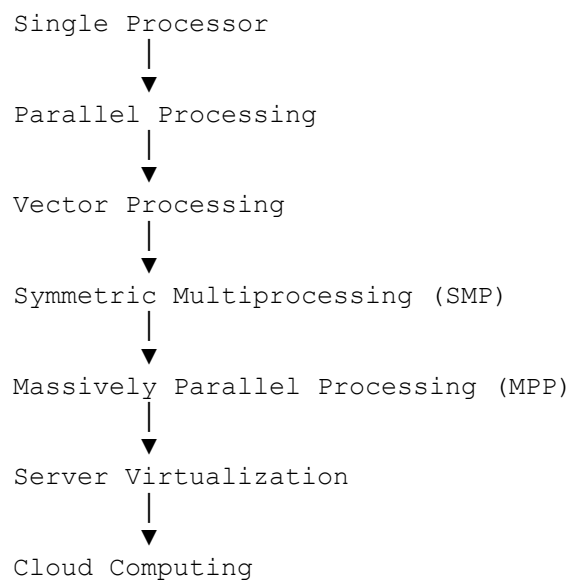
To overcome these limitations, several high-performance computing architectures evolved.

They include:

- Parallel Processing
- Vector Processing
- Symmetric Multiprocessing (SMP)
- Massively Parallel Processing (MPP)

These technologies laid the foundation for **server virtualization**, where a single physical server efficiently hosts multiple virtual machines.

## Evolution of Server Processing



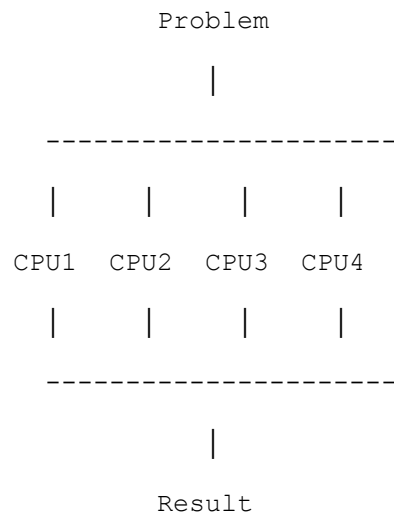
## 2. Parallel Processing

### Definition

Parallel processing is a computing technique in which **multiple processors execute different parts of a program simultaneously** to reduce execution time.

Instead of performing tasks one after another (sequential execution), several tasks are processed at the same time.

# Architecture



## Characteristics

- Multiple processors
- Simultaneous execution
- Faster computation
- Shared workload
- Reduced processing time

## Advantages

- High performance
- Faster execution
- Better CPU utilization
- Suitable for scientific applications

## Disadvantages

- Expensive hardware
- Complex programming
- Synchronization issues
- Difficult debugging

## Applications

- Artificial Intelligence
- Weather prediction
- Image processing
- Machine Learning
- Scientific simulations

## Example

Suppose a task requires 1000 calculations.

### Sequential Processing

CPU

↓

1000 Operations

↓

100 Seconds

### Parallel Processing

CPU1 → 250 Operations

CPU2 → 250 Operations

CPU3 → 250 Operations

CPU4 → 250 Operations

↓

25 Seconds

## 3. Vector Processing

### Definition

Vector processing is a computing technique where **a single instruction operates on multiple data elements simultaneously**.

Instead of processing one value at a time, vector processors handle an entire array (vector) of data in one operation.

# Scalar vs Vector Processing

## Scalar Processing

A1 + B1

A2 + B2

A3 + B3

A4 + B4

Each operation is executed separately.

## Vector Processing

[A1 A2 A3 A4]

+

[B1 B2 B3 B4]

↓

[C1 C2 C3 C4]

All operations are executed together.

## Characteristics

- Single instruction
- Multiple data items
- High-speed arithmetic
- Efficient for numerical computations

## Advantages

- Very fast mathematical computation
- Efficient for matrix operations
- Reduced execution time
- Better hardware utilization

## Disadvantages

- Suitable only for vectorizable problems
- Expensive hardware
- Limited general-purpose applications

## Applications

- Weather forecasting
- Engineering simulations
- Computer graphics
- Signal processing
- Deep Learning

# Difference Between Parallel and Vector Processing

| Parallel Processing                              | Vector Processing                                                   |
|--------------------------------------------------|---------------------------------------------------------------------|
| Multiple processors execute tasks simultaneously | One processor performs the same operation on multiple data elements |
| Multiple instructions                            | Single instruction on multiple data                                 |
| General-purpose computing                        | Numerical computations                                              |
| Uses several CPUs or cores                       | Uses vector processing units (SIMD)                                 |

## 4. Symmetric Multiprocessing (SMP)

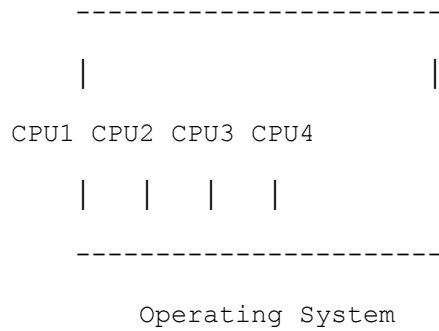
### Definition

Symmetric Multiprocessing (SMP) is a computer architecture in which **two or more identical processors share the same main memory and operating system.**

Each processor has equal access to all resources.

### Architecture

Shared Memory



## Characteristics

- Multiple identical processors
- Shared memory
- Single operating system
- Equal processor access
- Dynamic task scheduling

## How SMP Works

When multiple applications are running:

- Processor 1 executes Task A
- Processor 2 executes Task B
- Processor 3 executes Task C
- Processor 4 executes Task D

The operating system automatically distributes the workload.

## Advantages

- Improved performance
- Better load balancing
- Shared memory simplifies communication
- Higher system reliability
- Easy resource sharing

## Disadvantages

- Memory contention
- Scalability limitations

- Shared memory bottleneck
- Cost increases with processors

## Applications

- Enterprise servers
- Database servers
- Web servers
- Virtualization hosts
- Cloud servers

## Example

Modern Intel Xeon and AMD EPYC servers often use multiple CPU sockets with many cores, allowing the operating system to distribute workloads efficiently. These systems are widely used as hosts for virtual machines in cloud data centers.

# 5. Massively Parallel Processing (MPP)

## Definition

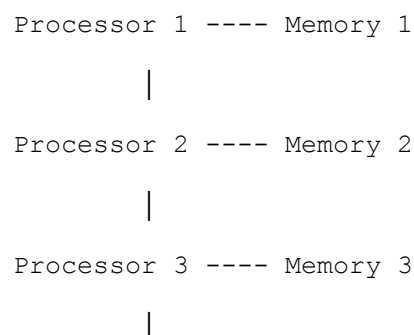
Massively Parallel Processing (MPP) is an architecture in which **hundreds or thousands of independent processors work together to solve extremely large computational problems.**

Each processor has:

- Its own CPU
- Its own memory
- Its own operating system resources

Processors communicate through a high-speed network.

## Architecture



Processor 4 ---- Memory 4

|

High-Speed Interconnection Network

## Characteristics

- Hundreds or thousands of processors
- Distributed memory
- Independent processing
- High scalability
- Fault tolerance

## Advantages

- Extremely high performance
- Excellent scalability
- Handles very large datasets
- Supports Big Data analytics

## Disadvantages

- Expensive infrastructure
- Complex programming
- Network communication overhead
- Difficult system management

## Applications

- Big Data processing
- Scientific computing
- Space research
- Artificial Intelligence
- Cloud data centers
- Financial modeling

## Examples

- Large cloud data centers
- Google Search infrastructure
- High-performance computing (HPC) clusters
- Scientific supercomputers

# SMP vs MPP

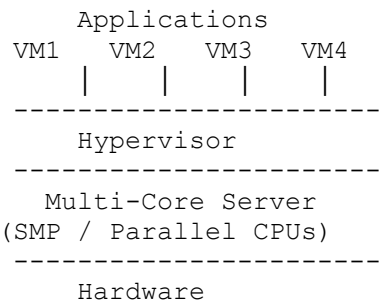
| Feature              | SMP                      | MPP                                      |
|----------------------|--------------------------|------------------------------------------|
| Memory               | Shared                   | Distributed                              |
| Operating System     | Single OS                | Independent processing nodes             |
| Number of Processors | Few (2–64 or more cores) | Hundreds or thousands                    |
| Scalability          | Moderate                 | Very High                                |
| Communication        | Shared memory            | Network messages                         |
| Cost                 | Lower                    | Higher                                   |
| Best Use             | Enterprise servers       | Supercomputers and large cloud platforms |

## Role of These Technologies in Server Virtualization

Server virtualization allows **multiple virtual machines (VMs)** to run on a single physical server. This became practical because of advances in high-performance server architectures.

| Technology          | Contribution to Virtualization                                     |
|---------------------|--------------------------------------------------------------------|
| Parallel Processing | Faster execution of multiple virtual machines simultaneously       |
| Vector Processing   | Accelerates AI, graphics, and scientific workloads within VMs      |
| SMP                 | Multiple processors/cores efficiently share workloads among VMs    |
| MPP                 | Enables large-scale distributed cloud services and data processing |

## How Server Virtualization Works



The **hypervisor** allocates CPU cores, memory, storage, and networking resources from the physical server to each virtual machine.

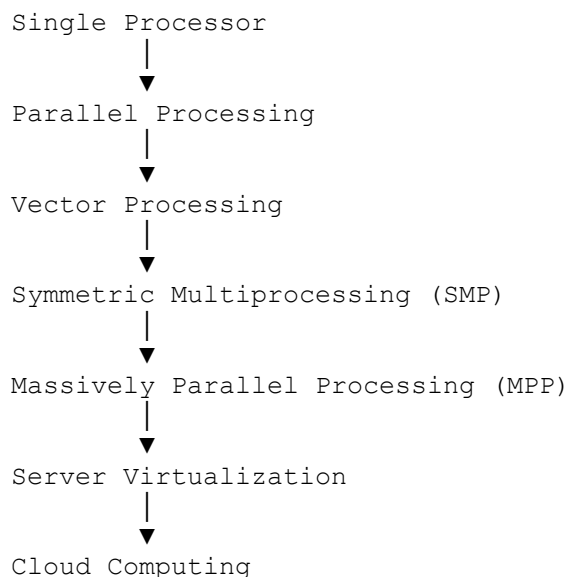
## Relationship to Cloud Computing

Modern cloud providers such as AWS, Microsoft Azure, and Google Cloud use:

- Multi-core SMP servers
- Parallel processing
- Vector instruction sets (SIMD/AVX) for AI workloads
- Large MPP clusters for analytics and distributed computing
- Hypervisors (VMware ESXi, Hyper-V, KVM, Xen) to create virtual machines

These technologies together enable Infrastructure as a Service (IaaS), where users can provision virtual servers on demand.

## Summary



## Topic 5 Web Services Overview

### 1. Introduction to Web Services

#### What is a Web Service?

A **Web Service** is a software system that enables communication and data exchange between different applications over the Internet using standard protocols.

A web service allows applications developed in different programming languages and running on different operating systems to communicate seamlessly.

#### Definition

**A Web Service is a standardized way of integrating web-based applications using open Internet protocols such as HTTP and XML.**

# Why Do We Need Web Services?

Different organizations use different technologies.

For example:

- Bank → Java
- Shopping Website → .NET
- Mobile App → Android
- Payment Gateway → Python

These applications must communicate with one another.

Web Services provide a common communication platform.

## Example

When purchasing a product online:

Customer

|

Shopping Website

|

Payment Gateway

|

Bank Server

|

Confirmation Service

Each system may use different programming languages but communicates through Web Services.

## Characteristics of Web Services

- Platform independent
- Language independent
- Interoperable
- Reusable
- Loosely coupled
- Accessible over the Internet
- Uses standard communication protocols

# Components of Web Services

Client Application

|

Request

|

Web Service

|

Business Logic

|

Database

|

Response

## Web Service Architecture

Service Provider

|

Publishes Service

|

UDDI

|

Client Searches

|

Client Gets Service Description

|

SOAP Request

|

HTTP

|

Service Provider

|

SOAP Response

The three major components are:

- Service Provider
- Service Registry (UDDI)
- Service Consumer (Client)

## 2. SOAP (Simple Object Access Protocol)

### Definition

SOAP is an XML-based messaging protocol used for exchanging structured information between applications over a network.

SOAP enables communication between distributed applications regardless of their implementation platform.

### Characteristics

- XML-based
- Platform independent
- Language independent
- Extensible
- Secure
- Supports distributed computing

### Features

- Standardized communication
- Built-in error handling
- Supports security standards (WS-Security)
- Supports transactions
- Reliable messaging

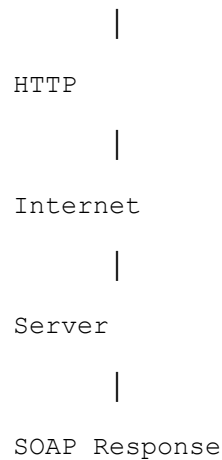
### SOAP Architecture

Application

|

SOAP Message

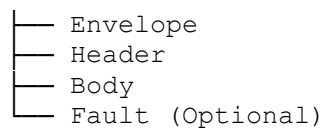
(XML)



## Structure of a SOAP Message

A SOAP message is an XML document containing four parts.

SOAP Message



## SOAP Envelope

Defines the beginning and end of the SOAP message.

Example

```
<Envelope>
...
</Envelope>
```

## SOAP Header

Contains optional information.

Examples:

- Authentication
- Security
- Transaction information

## SOAP Body

Contains the actual request or response.

Example

```
GetStudentDetails
```

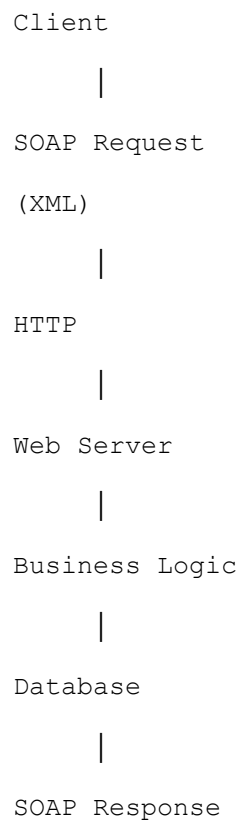
## SOAP Fault

Used for reporting errors.

Example

```
Invalid User
```

## SOAP Request Flow



## Advantages of SOAP

- High security

- Reliable communication
- Platform independent
- Standardized protocol
- Supports enterprise applications

## Disadvantages of SOAP

- XML messages are large
- Slower than REST
- Complex implementation
- Higher bandwidth usage

## Applications

- Banking systems
- Healthcare systems
- Enterprise Resource Planning (ERP)
- Airline reservation systems
- Government applications

## 3. HTTP (Hypertext Transfer Protocol)

### Definition

HTTP is an application-layer protocol used to transfer web pages and Web Service messages between clients and servers.

SOAP commonly uses HTTP as its transport protocol.

## Client–Server Communication

Browser

|

HTTP Request

|

Web Server

|

HTTP Response

|

Browser

## HTTP Request Methods

| Method | Purpose       |
|--------|---------------|
| GET    | Retrieve data |
| POST   | Send data     |
| PUT    | Update data   |
| DELETE | Remove data   |

## HTTP Request Structure

GET /student

Host: example.com

## HTTP Response Structure

Status Code

Headers

Message Body

## Common HTTP Status Codes

| Code | Meaning               |
|------|-----------------------|
| 200  | OK                    |
| 201  | Created               |
| 400  | Bad Request           |
| 401  | Unauthorized          |
| 404  | Not Found             |
| 500  | Internal Server Error |

# Advantages of HTTP

- Simple
- Fast
- Widely supported
- Platform independent
- Easy integration

## Role of HTTP in Web Services

HTTP acts as the transport mechanism that carries SOAP messages between the client and the service provider.

SOAP Message

↓

HTTP

↓

Internet

↓

Server

## 4. UDDI (Universal Description, Discovery and Integration)

### Definition

UDDI is a platform-independent directory service that enables businesses to publish, discover, and locate Web Services.

It acts as a **service registry**.

### Purpose of UDDI

Imagine an online telephone directory.

Instead of finding people, UDDI helps applications find available Web Services.

# UDDI Architecture

Service Provider

|

Publishes Service

|

UDDI Registry

|

Client Searches

|

Finds Service

|

Invokes Service

## Components of UDDI

### 1. Business Information

Contains organization details.

Examples:

- Company name
- Address
- Contact details

### 2. Service Information

Lists the services provided.

Example:

Payment Service

Student Service

Inventory Service

### 3. Technical Information

Provides technical details required to invoke the service.

Examples:

- Service endpoint
- Protocols
- Service description

## UDDI Operations

| Operation | Description              |
|-----------|--------------------------|
| Publish   | Register a Web Service   |
| Find      | Search for a Web Service |
| Bind      | Connect to the service   |
| Invoke    | Use the service          |

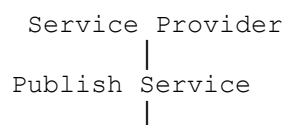
## Advantages of UDDI

- Easy service discovery
- Service reuse
- Better integration
- Platform independent
- Supports enterprise applications

## Disadvantages of UDDI

- Complex implementation
- Less commonly used in modern REST-based systems
- Requires maintenance of service registry

## Complete Working of SOAP, HTTP, and UDDI





## Relationship Between SOAP, HTTP, and UDDI

| Technology | Purpose                                                   |
|------------|-----------------------------------------------------------|
| SOAP       | Defines the message format exchanged between applications |
| HTTP       | Transports SOAP messages across the Internet              |
| UDDI       | Helps clients discover and locate available Web Services  |

## Role of Web Services in Cloud Computing

Cloud platforms expose services through Web APIs and Web Services.

Examples include:

- Authentication Service
- Payment Service
- Storage Service
- Database Service
- Email Service

In modern cloud environments:

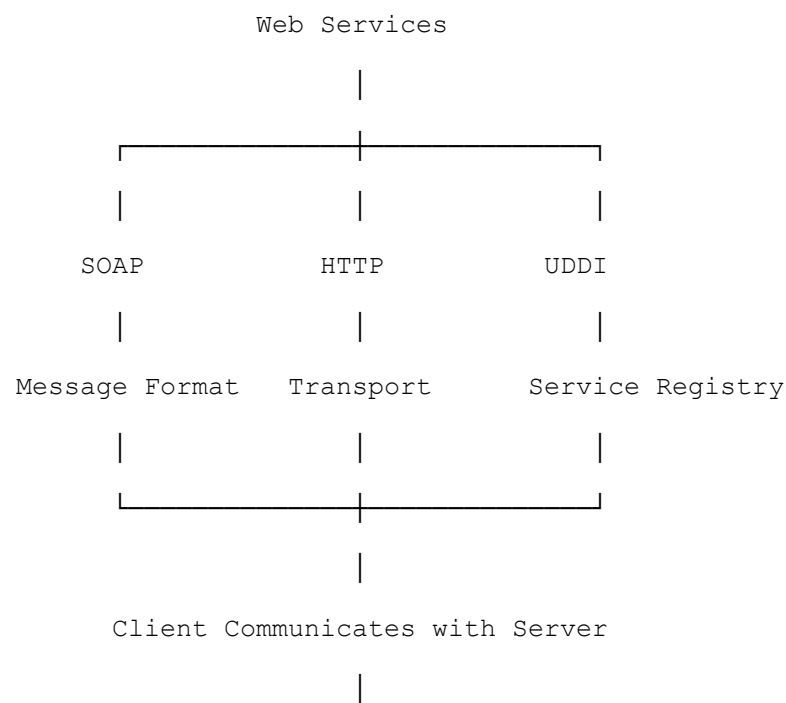
- **SOAP** is still used in many enterprise applications that require high security and reliability.
- **HTTP/HTTPS** is the standard protocol for communication between clients and cloud services.

- **UDDI** introduced the concept of service discovery, which has evolved into modern API gateways and service registries used in cloud-native architectures.

# Comparison of SOAP, HTTP, and UDDI

| Feature     | SOAP                               | HTTP                                | UDDI                                             |
|-------------|------------------------------------|-------------------------------------|--------------------------------------------------|
| Full Form   | Simple Object Access Protocol      | Hypertext Transfer Protocol         | Universal Description, Discovery and Integration |
| Type        | Messaging Protocol                 | Communication Protocol              | Service Registry                                 |
| Purpose     | Exchange structured messages       | Transfer messages over the Internet | Discover and publish Web Services                |
| Data Format | XML                                | Any format (HTML, XML, JSON, etc.)  | XML-based service information                    |
| Role        | Communication between applications | Transport mechanism                 | Service discovery                                |

## Summary



## Summary

- A **Web Service** enables communication between different applications over a network.
- **SOAP** defines how messages are formatted and exchanged using XML.
- **HTTP** transports these messages between clients and servers.
- **UDDI** acts as a directory for publishing and discovering Web Services.
- Together, SOAP, HTTP, and UDDI formed the foundation of traditional Service-Oriented Architecture (SOA) and influenced the development of cloud-based services.

## Topic 6 Cloud Service Models (IaaS, PaaS, SaaS and XaaS)

### 1. Introduction

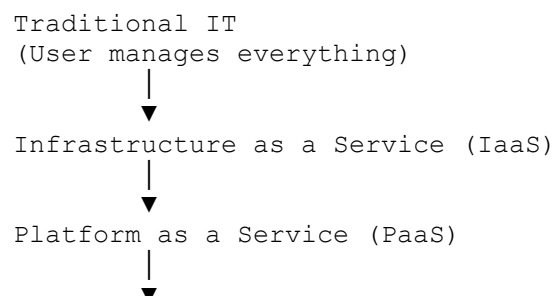
Cloud computing provides computing resources as **services** over the Internet instead of requiring organizations to purchase and maintain their own infrastructure.

These services are broadly classified into:

- Infrastructure as a Service (IaaS)
- Platform as a Service (PaaS)
- Software as a Service (SaaS)
- Everything as a Service (XaaS)

Each model provides a different level of responsibility to the cloud provider and the customer.

### Evolution of Cloud Services



Software as a Service (SaaS)  
↓  
Everything as a Service (XaaS)

As we move downward:

- Customer management decreases.
- Cloud provider responsibility increases.

## Why Do We Need Cloud Service Models?

Organizations have different requirements.

For example:

- A software developer needs servers and networking.
- A web developer needs only a programming platform.
- A student needs only software.
- An enterprise may require AI services, databases, storage, and networking.

Cloud service models provide appropriate solutions for each need.

## 2. Infrastructure as a Service (IaaS)

### Definition

Infrastructure as a Service (IaaS) provides **virtualized computing resources such as servers, storage, and networking** over the Internet.

The cloud provider manages the physical infrastructure, while the customer manages the operating system, applications, and data.

### Architecture

Applications  
|  
Operating System  
|  
Virtual Machines  
|  
Storage  
|  
Networking  
|  
Physical Servers

The provider owns and maintains the physical infrastructure.

## **Provider Responsibilities**

- Physical servers
- Storage devices
- Networking
- Virtualization
- Data center maintenance

## **Customer Responsibilities**

- Operating system
- Software installation
- Applications
- Databases
- Security configuration
- Data

## **Characteristics**

- On-demand virtual machines
- Scalable infrastructure
- Pay-as-you-go pricing
- High availability
- Virtualized resources

## **Advantages**

- No hardware purchase
- Flexible scaling
- Lower infrastructure cost
- Faster deployment
- Full control over the operating system

## **Disadvantages**

- Customer manages software updates
- Security configuration is the customer's responsibility
- Requires system administration skills

## **Examples**

- Amazon EC2
- Microsoft Azure Virtual Machines
- Google Compute Engine
- DigitalOcean Droplets

## Real-Life Analogy

### Renting an Empty House

The landlord provides:

- Building
- Electricity
- Water

The tenant brings:

- Furniture
- Appliances
- Decoration

Similarly,

Cloud provider provides infrastructure.

Customer installs software.

## Applications

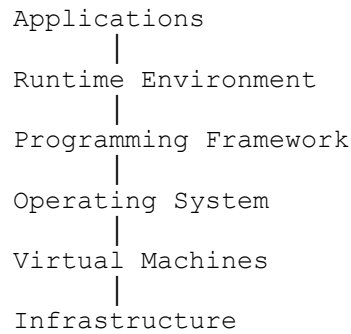
- Website hosting
- Enterprise applications
- Disaster recovery
- Development environments
- Backup servers

## 3. Platform as a Service (PaaS)

### Definition

Platform as a Service (PaaS) provides a complete development platform where developers can build, test, and deploy applications without managing the underlying infrastructure.

## Architecture



## Provider Responsibilities

- Infrastructure
- Operating system
- Middleware
- Runtime
- Security patches
- Database services (in many cases)

## Customer Responsibilities

- Application code
- Business logic
- Application data

## Characteristics

- Ready-to-use development environment
- Automatic scaling
- Built-in development tools
- Supports multiple programming languages

## Advantages

- Faster application development
- No server management
- Automatic updates
- Reduced development cost
- Easy deployment

# Disadvantages

- Less control over infrastructure
- Vendor lock-in
- Platform restrictions

# Examples

- Google App Engine
- Microsoft Azure App Service
- Heroku
- Red Hat OpenShift

# Real-Life Analogy

## Renting a Fully Equipped Kitchen

The owner provides:

- Kitchen
- Gas
- Stove
- Utensils

You only cook.

Similarly,

The cloud provider manages the platform.

Developers focus on writing code.

# Applications

- Mobile applications
- Web applications
- API development
- Educational software

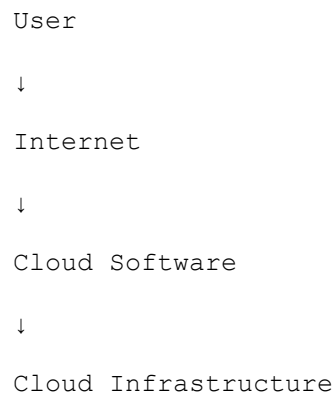
## 4. Software as a Service (SaaS)

### Definition

Software as a Service (SaaS) provides complete software applications over the Internet.

Users simply access the software through a web browser or mobile application without installing or maintaining it.

### Architecture



### Provider Responsibilities

- Infrastructure
- Operating system
- Runtime
- Database
- Application
- Security
- Updates
- Maintenance

### Customer Responsibilities

- User account
- Application usage
- Data entered by the user

# Characteristics

- Ready-to-use software
- Subscription model
- Accessible anywhere
- Automatic updates
- Browser-based access

# Advantages

- No installation
- Low maintenance
- Easy accessibility
- Automatic updates
- Lower initial cost

# Disadvantages

- Limited customization
- Internet dependency
- Vendor lock-in
- Data privacy concerns

# Examples

- Gmail
- Microsoft 365
- Google Docs
- Salesforce
- Zoom
- Dropbox

# Real-Life Analogy

## Watching Movies on Netflix

You do not purchase DVDs.

You simply subscribe and use the service.

Similarly,

Users subscribe and use software without worrying about installation or maintenance.

# Applications

- Email
- Video conferencing
- Customer Relationship Management (CRM)
- Office productivity
- Online education

## 5. Everything as a Service (XaaS)

### Definition

Everything as a Service (XaaS) is a cloud computing model where **almost any IT resource can be delivered as a service over the Internet**.

XaaS extends the concepts of IaaS, PaaS, and SaaS to include specialized services.

### Why XaaS?

Organizations increasingly require more than just infrastructure or software.

Examples include:

- Artificial Intelligence
- Databases
- Storage
- Security
- Containers
- Networking

These services are delivered on demand through XaaS.

## Common Types of XaaS

### Storage as a Service (STaaS)

Provides cloud storage.

Examples:

- Amazon S3
- Google Cloud Storage
- Azure Blob Storage

## **Database as a Service (DBaaS)**

Provides managed databases.

Examples:

- Amazon RDS
- Azure SQL Database
- Google Cloud SQL

## **Function as a Service (FaaS)**

Runs small pieces of code without managing servers.

Examples:

- AWS Lambda
- Azure Functions
- Google Cloud Functions

## **Network as a Service (NaaS)**

Provides networking resources as a service.

Examples:

- Virtual Private Network (VPN)
- Cloud networking
- SD-WAN

## **Security as a Service (SECaaS)**

Provides security solutions.

Examples:

- Cloud antivirus
- Identity management
- Firewall services

## **Container as a Service (CaaS)**

Provides container deployment and management.

Examples:

- Google Kubernetes Engine (GKE)
- Azure Kubernetes Service (AKS)
- Amazon Elastic Kubernetes Service (EKS)

## **Artificial Intelligence as a Service (AIaaS)**

Provides AI capabilities through cloud APIs.

Examples:

- Speech recognition
- Image recognition
- Machine learning APIs
- Chatbots

## **Characteristics**

- Highly scalable
- Subscription-based
- Pay-per-use
- Easy integration
- Flexible deployment

## **Advantages**

- Reduces operational costs
- Faster deployment
- Easy scalability
- Latest technologies available on demand

- No infrastructure management

## Disadvantages

- Internet dependency
- Vendor lock-in
- Security concerns
- Compliance issues

## Comparison of IaaS, PaaS, SaaS, and XaaS

| Feature                   | IaaS                        | PaaS                          | SaaS                  | XaaS                         |
|---------------------------|-----------------------------|-------------------------------|-----------------------|------------------------------|
| Full Form                 | Infrastructure as a Service | Platform as a Service         | Software as a Service | Everything as a Service      |
| User Control              | High                        | Medium                        | Low                   | Depends on the service       |
| Infrastructure Managed By | Provider                    | Provider                      | Provider              | Provider                     |
| Operating System          | Customer                    | Provider                      | Provider              | Provider                     |
| Application Management    | Customer                    | Customer                      | Provider              | Provider or Customer         |
| Target Users              | System Administrators       | Developers                    | End Users             | Organizations and Developers |
| Pricing                   | Pay-as-you-use              | Subscription/Usage            | Subscription          | Pay-per-use or Subscription  |
| Examples                  | EC2, Azure VM               | App Engine, Azure App Service | Gmail, Microsoft 365  | AWS Lambda, Amazon RDS, GKE  |

## Responsibility Model

| Component        | IaaS     | PaaS     | SaaS     |
|------------------|----------|----------|----------|
| Applications     | Customer | Customer | Provider |
| Data             | Customer | Customer | Customer |
| Runtime          | Customer | Provider | Provider |
| Middleware       | Customer | Provider | Provider |
| Operating System | Customer | Provider | Provider |
| Virtualization   | Provider | Provider | Provider |
| Servers          | Provider | Provider | Provider |

|            |          |          |          |   |
|------------|----------|----------|----------|---|
| Storage    | Provider | Provider | Provider |   |
| Networking | Provider | Provider | Provider |   |
| +-----+    | +-----+  | +-----+  | +-----+  | + |

# Real-World Case Study

A startup wants to launch an online shopping application.

## Option 1 – IaaS

- Rent virtual machines.
- Install Linux, web server, and database.
- Manage updates and security.

**Suitable when:** Full control over the infrastructure is required.

## Option 2 – PaaS

- Write the application code.
- Deploy directly to the cloud platform.
- The provider manages servers and runtime.

**Suitable when:** The focus is on application development.

## Option 3 – SaaS

- Use an existing e-commerce platform.
- Simply create an account and configure the store.

**Suitable when:** Quick deployment with minimal technical effort is desired.

## Option 4 – XaaS

- Use AI services for recommendations.
- Use DBaaS for managed databases.
- Use STaaS for file storage.
- Use FaaS for event-driven order processing.

**Suitable when:** Specialized cloud services are needed to enhance the application.

# Summary

Cloud Computing

